In VBA, interacting with rows, columns, and sheets is done by referencing specific **Objects.** Because Excel supports up to **1,048,576 rows** and **16,384 columns** per sheet, using efficient referencing methods is key to building fast macros.

---

1. Interacting with Sheets

You can reference worksheets by their name, their index number, or their "CodeName."

- **By Name**: `Worksheets("Salary statement").Activate` — Best for readability.
- **By Index**: `Worksheets(1).Select` — References the first sheet on the left.
- **Adding/Deleting**:
    - `Worksheets.Add After:=Worksheets(Worksheets.Count)` — Adds a new sheet at the end.
    - `Worksheets("Sheet1").Delete` — Removes a specific sheet.
- **The "With" Block**: To perform multiple actions on one sheet without repeating its name:
- VBA

```
With Worksheets("Sales Data")
    .Range("A1").Value = "Report"
    .Columns("A:B").AutoFit
End With
```

-
-

---

2. Interacting with Rows and Columns

Rows and Columns can be manipulated as entire entities for formatting or data cleanup.

Rows

- **Select/Reference**: `Rows(5).Select` or `Rows("5:10").Hidden = True`.
- **Insert/Delete**: `Rows(2).Insert` — Shifts data down to add a new record.
- **Height**: `Rows("1:5").RowHeight = 25`.

Columns

- **Select/Reference**: `Columns("B").Select` or `Columns(2).Select`.
- **AutoFit**: `Columns("A:Z").AutoFit` — Automatically adjusts widths to fit the longest text, such as a "Student Name".
- **Visibility**: `Columns("C").Hidden = True`.

---

3. Dynamic Referencing (Finding the Last Row)

When processing datasets like a "Sales chart" or "Salary list," you often don't know how many rows are filled. Hard-coding a range is risky; instead, use this common VBA "hack" to find the last used row dynamically:

```
VBA

Dim lastRow As Long
lastRow = Cells(Rows.Count, "A").End(xlUp).Row
```

- **Logic**: This starts at the very bottom of the sheet (Row 1,048,576) and "jumps" up until it hits the first cell with data.

---

4. The Range vs. Cells Property

Method	Syntax	Best Use Case
Range	`Range("A1:B10")`	Working with fixed blocks of cells or named ranges.
Cells	`Cells(row, col)`	Using variables in **Loops** (e.g., `Cells(i, 1)`).

---

5. Pro-Tips for Speed

- **Avoid `.Select` and `.Activate`**: You don't need to physically "select" a cell to change it. `Range("A1").Value = 50` is much faster than `Range("A1").Select` followed by `ActiveCell.Value = 50`.
- **Toggle Screen Updating**: When looping through many rows, use `Application.ScreenUpdating = False` at the start to prevent the screen from flickering and to speed up execution.
- **Clear vs. Delete**: Use `.ClearContents` to keep the formatting but remove the data; use `.Delete` to remove the cells entirely and shift surrounding data.